Haskell is Bullshit

Typeclasses

Haskell defines typeclasses such as *Num*, *Ord* etc which encapsulate common patterns across types. For example, we know that any type in the class *Num* has arithmetic operations (+, *, etc) defined, meaning we can define functions that are of type "*Num a*" instead of having to provide a separate function definition for each type (Int, Integer, Float, …) that we want our function to support.

Without Typeclasses:

```
doubleme :: Int -> Int
doubleme x = x + x
ghci> doubleme 5
10
ghci> doubleme 1.2

<interactive>:3:10: error: [GHC-39999]
```

We would need a separate definition of *doubleme* for Float (and any other numeric types we would like to support)

With Typeclasses:

```
doubleme :: Num a => a -> a
doubleme x = x + x
ghci> doubleme 5
10
ghci> doubleme 1.2
2.4
```

Here, *a* represents any type satisfying the constraint (*a* is an instance of typeclass *Num*), specifying that the input and output are both of the same type *a*. This function works with any *a* satisfying the constraint, i.e. all numeric types.

Haskell Typeclasses:

- *Eq* - types supporting equality/inequality
- *Ord* - types supporting ordering, i.e. <, >
- *Read* - types that can be read from the I/O
- *Show* - types that can be printed
- *Num* - numerical types that allow addition and multiplication
- *Functor*, *Applicative*, *Monad* - ???

```
class StupidType a where
    isStupid :: a -> Bool

instance StupidType Int where
    isStupid :: Int -> Bool
    isStupid _ = False

instance StupidType Char where
    isStupid :: Char -> Bool
    isStupid _ = True
ghci> isStupid 'a'
True
```

We can also define our own typeclasses and make existing types instances of them.

To make a type an instance of a typeclass, we must provide a definition of all the required methods for that typeclass - e.g. for *Eq* we must define == . We could make a *String* a *Num* if we wanted to, we just need to define how the necessary methods work on Strings.

```
instance Num String where
    (+) :: String -> String -> String
    (+) = undefined
    (*) :: String -> String -> String
    (*) = undefined
```

Functors

The *Functor* typeclass is a class of types which can be mapped over, using a function called *fmap*. This means that all type instances of *Functor* must define a function *fmap* as follows:

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

Let's consider *List*s as an example.

A *List* is an instance of the typeclass *Functor*, with its version of *fmap* being the standard *map* function, which has the following type:

```
map :: (a -> b) -> [a] -> [b]
```

Here, *f a* and *f b* are replaced with *[a]* and *[b]*. Basically, the *f* in the general *fmap* definition represents the type of Functor and *a* and *b* represent the types of the contents of the Functor.

Generally, any type that acts as a "container" for items of some other type is a Functor. This includes things like *Maybe*, which encapsulates an item of some type as a *Just* (or a *Nothing* if it is not there). For *Maybe*, *fmap* is defined as follows:

```
instance Functor Maybe where
    fmap f (Just x) = Just (f x)
    fmap f Nothing = Nothing
```

All *fmap* implementations should obey the Functor Laws:

- *fmap id c == c*
- *fmap f (fmap g c) == fmap (f . g) c*

i.e. Mapping the identity function *id* over a container should return the same container with its contents unmodified, and performing two maps, with some function *f* and then with some other function *g* is the same as performing a single map with the composition of *f* and *g*.

These laws are not enforced by the compiler, but following them ensures that all Functors behave predictably, avoiding unwanted side-effects.

Applicatives

Functors allow us to map single-argument functions to “containers” of “things”, using *fmap*.

```
ghci> fmap (+1) (Just 1)
Just 2
```

But what if we want to add two *Maybe Int*s together? We cannot do *(Just 1) + (Just 2)* for example as *+* isn't defined for *Maybes* (just as *[1] + [1]* is undefined, this is the same across all *Functor*s of *Num*s). We use *Applicative*s to map functions with any number of arguments over a container. The *Applicative* typeclass is a subclass of *Functor*, defined as follows:

```
class (Functor f) => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b
```

pure takes in a value of any type (including functions) and returns an applicative functor containing that value - i.e. it “wraps the value into a container”.

<*> applies a function within an applicative such that all inputs and the output are contained within the applicative.

Now, if we wanted to add *Just 1* to *Just 1*:

```
ghci> pure (+) <*> Just 1 <*> Just 1
Just 2
```

Why does this work? *pure (+)* produces a function of type *(Applicative f, Num a) => f (a -> a -> a)*. The <*> operation is left-associative, so because of currying we first apply *Just 1* to this function, resulting in a partially applied function of type *Num a => Maybe (a -> a)*. *Maybe* here is the applicative that takes the place of the general placeholder *f* in the definition. Applying the second *Just 1* gives us the value *Just 2* which is of type *Num a => Maybe a*.

Basically, <*> takes a function and a value both encapsulated in the same applicative functor, unwraps the functor, applies the function to the value and rewraps the result into the same functor. *Pure* is used to wrap something, e.g. the *(+)* function (or any “normal” function), into the same applicative to allow this application to take place.

pure f <> x* is equivalent to *fmap f x*. In Haskell, we have <$>, which is just

fmap defined as an infix operator. This means that we can rewrite our adding of two *Just 1*s as:

```
ghci> (+) <$> Just 1 <*> Just 1
Just 2
```

Note: the *Applicative* instance definition of *Maybe* is as follows:

```
instance Applicative Maybe where
    pure = Just
    Nothing <*> _ = Nothing
    (Just f) <*> something = fmap f something
```

*List*s are also *Applicative*s, defined as:

```
instance Applicative [] where
    pure x = [x]
    fs <*> xs = [f x | f <- fs, x <- xs]
```

The list comprehension used in the definition for <*> means that the result of applying a list of applicative functions (*pure x = [x]* but nothing is stopping this list *[x]* from containing multiple functions) to a list of items produces all possible combinations, for example:

```
ghci> [(+1),(+2)] <*> [1,2,3,4]
[2,3,4,5,3,4,5,6]
```

This extends to functions taking multiple arguments:

```
ghci> [(+),(*)] <*> [1,2,3,4] <*> [5,6,7]
[6,7,8,7,8,9,8,9,10,9,10,11,5,6,7,10,12,14,15,18,21,20,24,28]
```

Holy shit they stack

```
ghci> [(pure (+2)<*>)] <*> [Just 1, Nothing, Just 67]
[Just 3,Nothing,Just 69]
```

^ This applies the function *(pure (+2) <*>)* to all values in the list. This could also have been done with *fmap*.

```
ghci> fmap (fmap (+2)) [Just 1, Nothing, Just 67]
[Just 3,Nothing,Just 69]
```

The definitions of *pure* and <*> for all *Applicative*s should satisfy the Applicative Laws:

- *pure id <*> x = x* - Applying the identity function id to a value in an applicative context does not change the value
- *pure (g x) = pure g <*> pure x* - Applying a pure function to a pure value in an applicative context is the same as applying the function to the value normally, then wrapping the result.
- *x <*> pure y = pure (\g -> g y) <*> x* - pointless waffle
- *x <*> (y <*> z) = (pure (.) <*> x <*> y) <*> z* - Basically (x.y) z = x (y z)

Monads

The *Monad* typeclass is a subclass of *Applicative*, defined as:

```
class Applicative m => Monad m where
    return :: a -> m a

    (>>=) :: m a -> (a -> m b) -> m b

    (>>) :: m a -> m b -> m b
    x >> y = x >>= \_ -> y
```

return is exactly the same as *pure* with Applicatives. The *>>=* "bind" operator applies a value encapsulated in some Monad *m*, "unwraps" it and applies it to a function that takes a normal (non-encapsulated) value and returns a monadic ("wrapped") value. The *>>* operator does the same thing but without the value passing, and is usually not implemented in specific instances as it has a default implementation.

An example instance of the *Monad* typeclass is *Maybe*, defined as:

```
instance Monad Maybe where
    return x = Just x
    Nothing >>= f = Nothing
    Just x >>= f  = f x
```

Basically, if we feed *Nothing* into a function the result is always *Nothing*, but if we apply *Just x* to a function *f* it applies *f* to *x* (and since the definition of *>>=* states that *f* must be of type *a -> m b*, the result of this application must also be a *Maybe*)

```
ghci> let xten x = return (x * 10)
ghci> Just 67 >>= xten
Just 670
ghci> Nothing >>= xten
Nothing
```

We use *return* in the *xten* function so that it is of type *(Monad m, Num a) => a -> m a* as required by *>>=*

We can chain >>=s to create a monstrosity

```
isEven :: Integral a => a -> Maybe a
isEven a
    | even a = Just a
    | otherwise = Nothing

addThreeEvens :: Num a => a -> a -> a -> Maybe a
addThreeEvens a b c = Just (a + b + c)
```

```
ghci> isEven 4 >>= \a -> isEven 6 >>= \b -> isEven 8 >>= \c -> addThreeEvens a b c
Just 18
ghci> isEven 4 >>= \a -> isEven 6 >>= \b -> isEven 7 >>= \c -> addThreeEvens a b c
Nothing
```

Or, using *do* notation:

```
x :: Maybe Integer
x = do
    a <- isEven 4
    b <- isEven 6
    c <- isEven 8
    addThreeEvens a b c
```

Note: do notation is just syntax, it has no effect on the logic.

Monads should follow the Monad Laws:

- *return a >>= f = f a - Wrapping a value in a context and binding it to a function is the same as applying the function to the value*
- *m >>= return = m - binding a value wrapped in a Monad leaves the value unchanged.*
- *(m >>= (\x -> g x)) >>= (\y -> h y) = m >>= (\x -> g x >>= (\y -> h y)) - Associativity*

Foldables

The *Foldable* type class is the class of types that implement *foldr* and *foldl* - Note: the minimum definition only requires *foldr*.

Consider a *List* [1,2,3,4,5]. This can also be written as 1 : 2 : 3 : 4 : 5 : []. The function *foldr* is defined as follows:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []     = z
foldr f z (x:xs) = x `f` (foldr f z xs)
```

This is equivalent to performing *1 `f` (2 `f` (3 `f` (4 `f ( 5 `f` z ))))*, basically replacing the *:* operator in the list construction with *`f`* and replacing *[]* with the supplied value *z*. On the other hand *foldl* is defined as:

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z []     = z
foldl f z (x:xs) = foldl f (z `f` x) xs
```

Equivalent to *((((z `f` 1) `f` 2) `f` 3) `f` 4) `f` 5*. Note that *z* is the first argument of the first call of *f* in *foldl* but the second argument in the first call to *f* in *foldr*. For commutative operations, *foldl* and *foldr* produce the same results, however generally *foldr* is preferred as it works on infinite lists and can terminate early (apparently ?)

```
ghci> foldr (-) 10 [1]
-9
ghci> foldr (-) 10 [1,1]
10
ghci> foldr (-) 10 [1,1,1]
-9
ghci> foldr (-) 10 [1,1,1,1]
10
```

```
ghci> foldl (-) 10 [1]
9
ghci> foldl (-) 10 [1,1]
8
ghci> foldl (-) 10 [1,1,1]
7
ghci> foldl (-) 10 [1,1,1,1]
6
```

foldr vs *foldl* with subtraction. Because the result of the previous function call is passed to the second argument of the next one in *foldr* we get this goofy ahh result -> 1 - (1 - (1 - (1 - 10) = 10

Traversables

The *Traversable* typeclass contains types defining *traverse* (or *sequenceA*). This includes *List*s:

```
instance Traversable [] where
    -- sequenceA :: Applicative f => [f a] -> f [a]
    sequenceA []     = pure []
    sequenceA (u:us) = (:) <$> u <*> sequenceA us

sequenceA :: Applicative f => t (f a) -> f (t a)
```

It can be used for example to validate a list of *Maybe*s:

```
ghci> sequenceA [Just 67, Just 59]
Just [67,59]
ghci> sequenceA [Just 67, Just 59, Nothing]
Nothing
```

The *traverse* function is basically an upgrade of *fmap* that handles Applicatives.

For example, if we are performing an operation on a list of *Maybe*s and one of them fails (i.e. returns *Nothing*), then with *traverse* the result of the whole map will be *Nothing*.

```
safeDiv :: Integral a => a -> a -> Maybe a
safeDiv _ 0 = Nothing
safeDiv x y = Just (x `div` y)
ghci> traverse (safeDiv 8) [4,2]
Just [2,4]
ghci> traverse (safeDiv 8) [4,2,0]
Nothing
```